AXIO/1: Internal Languages, Categorical Models, and the Spectral Lattice of Formal Systems

Groupoid Infinity

May 30, 2026

Abstract

We analyze the Groupoid Infinity stack of formal languages as a *lattice* of internal languages, not a linear tower. The languages fall into three distinct branches: a *constructive type theory and set analysis* spine (Henk–Frank–Christine), an Homotopy Type Theory and Super Differential Smooth Quantum Spectra *super homotopy* spine (Henk–Per–Anders–Urs), and a *infinity category theory* spine (Dan–Mike–Ulrik). These spines are connected by functorial relationships but their syntactic term structures are fundamentally incompatible and should not be unified into a single AST. In particular, simplicial type theory (Ulrik/STT) is connected to Urs via directed-homotopy foundations, providing the simplicial backbone for its 3-layer architecture: Super-Smooth, Differential K-Theory, and Quantum Linear types. We construct syntactic CW-complexes and chain complexes for each language individually, define their cohomology groups, and assemble a spectral sequence over the full lattice of language categories.

Contents

1 Categorical Foundations

Definition 1.1 (Categories with Families (CwF))**.** A CwF is a category $\mathcal{C}$ of contexts and substitutions equipped with presheaves $Ty, Tm : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ and a representable natural transformation $p : Tm \to Ty$ satisfying context comprehension via pullback squares.

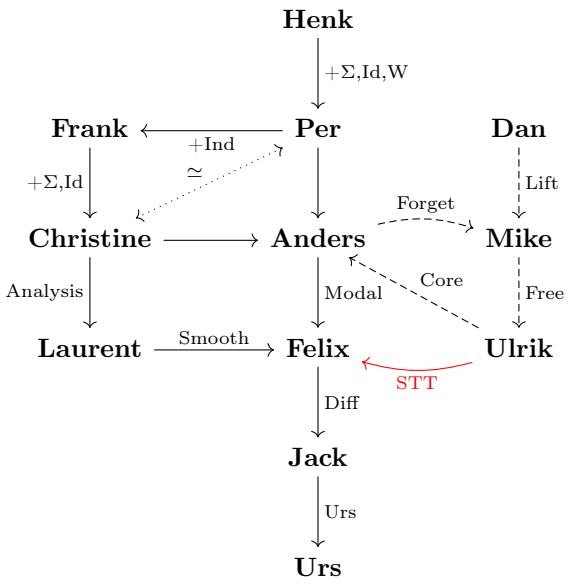
Definition 1.2 (Natural Models)**.** Following Awodey, a natural model is a category $\mathcal{C}$ with a terminal object $\mathbf{1}$ and a representable natural transformation $p : Tm \to Ty$. Pullbacks of p along Yoneda provide context extension and substitution.

Definition 1.3 (Grothendieck Fibration)**.** A functor $p : \mathcal{E} \to \mathcal{B}$ is a fibration when for every $X \in \mathcal{E}$ and $u : J \to p(X)$ in $\mathcal{B}$ there exists a Cartesian lift $f : Y \to X$ in $\mathcal{E}$.

The category of languages $\mathcal{L}$ has formal languages as objects and syntactic functors (compilers, type checkers, normalizers, extractors) as morphisms.

2 The Three Branches of the Language Lattice

The Groupoid Infinity stack does *not* form a linear tower. It decomposes into three distinct vertical columns: a constructive/inductive spine on the left (**Frank**–**Christine**–**Laurent**), a homotopical/cohesive spine in the center (**Henk**–**Per**–**Anders**–**Urs**), and an operational/directed branch on the right (**Dan**–**Mike**–**Ulrik**). The relationships between columns are functorial (lifts, free constructions, forgetful maps, smooth completions) but the syntaxes themselves are disjoint.

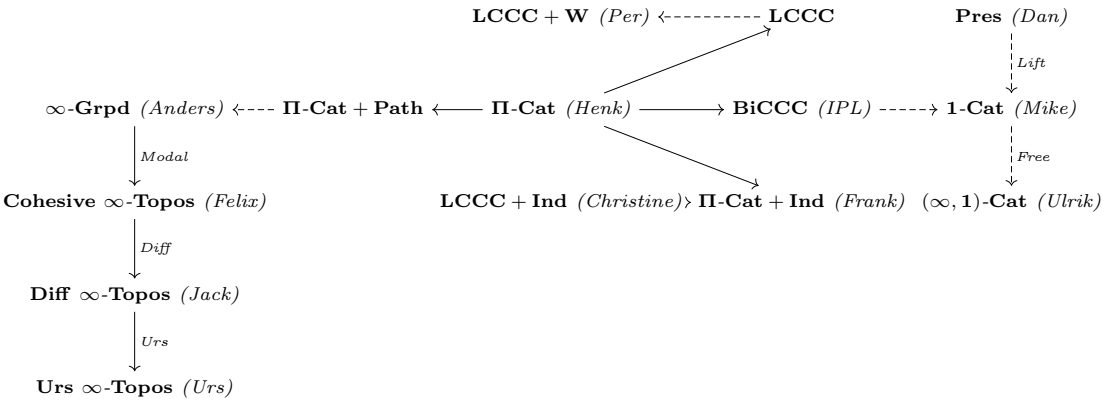


Remark 2.1 (Christine $\simeq$ Per)**.** Christine (full CIC with Σ, Id, Ind, Prop) and Per (MLTT-80 with Σ, Id, W, 0, 1, 2) are at a comparable level of expressiveness. Both serve as the foundation for Anders and can be considered interchangeable base systems. The choice between inductive schemes (Frank/Christine path) and W-types (Per path) is a design decision, not a strict ordering.

2.1 The Fine-Grained Primitive Lattice

To analyze the gradual expansion of language primitives, we construct a fine-grained lattice of semantic categories radiating from the category of dependent products (**Π-Cat** (Henk)). The system expands in four cardinal directions (adding dependent sums to yield locally cartesian closed categories **LCCC**, inductive algebras **Π-Cat** + **Ind** (Frank), path objects for identity types **Π-Cat** + **Path**, and bicartesian closed structures **BiCCC** (IPL)) before merging into rich categories of models such as **LCCC** + **Ind** (Christine), **LCCC** + **W** (Per), ∞-**Grpd** (Anders), and cohesive ∞-topoi. On the right, the lattice incorporates the operational algebra branch (**Pres** (Dan)), the directed 1-category branch (**1-Cat** (Mike)), and the simplicial $(\infty, 1)$-category branch ($(\infty, \mathbf{1})$-**Cat** (Ulrik)). All cross-branch connections are detailed in the global map in Section 2.

Theorem 2.2 (Universal Primitive Lattice)**.** *The categories of models for the Groupoid Infinity language stack form a planar, coherent lattice of categories radiating from the category of dependent products* **Π-Cat** *(Henk), extending in four cardinal directions (dependent sums, inductive types, path objects, and bicartesian closed structures), and completing in cohesive and directed higher categories:*



3 Branch I: Set Analysis

This branch represents the constructive/inductive spine of type theories and functional analysis. It is characterized by inductive algebraic constructions and real/functional analysis foundations.

3.1 Frank: Minimal CIC

Extends Henk with strictly positive inductive schemes.

```
type term =
    | Var of string | Universe of int
    | Pi of string * term * term | Lam of string * term * term | App of term * term
    | Inductive of inductive | Constr of int * inductive * term list
    | Ind of inductive * term * term list * term
and inductive = { name : string; params : (string * term * term) list;
    level : int; constrs : (int * term) list }
```

3.2 Christine: Full CIC (CwF/CwA)

Complete Calculus of Inductive Constructions with Σ, Id, and Ind.

```
type term =
    | Var of name | Universe of level
    | Pi of name * term * term | Lam of name * term * term | App of term * term
    | Sigma of name * term * term | Pair of term * term | Fst of term | Snd of term
    | Id of term * term * term | Refl of term
    | J of term * term * term * term * term * term
    | Inductive of inductive | Constr of int * inductive * term list
    | Ind of inductive * term * term list * term
and inductive = { name : string; params : (name * term * term) list;
    level : level; constrs : (int * term) list }
```

3.3 Laurent: Calculus and Functional Analysis

Laurent formalizes classical real and functional analysis, distribution theory, and measure theory. It provides first-class support for reals, complex numbers, measures, limits, and integration.

```
type exp_laurent =
  | Real | Complex | Nat | Bool
  | ConstReal of float | ConstComplex of float * float
  | Forall of string * exp_laurent * exp_laurent
  | Exists of string * exp_laurent * exp_laurent
  | Set of exp_laurent | Member of exp_laurent * exp_laurent
  | Measure of exp_laurent | Integral of exp_laurent * exp_laurent * exp_laurent
  | Seq of (int -> exp_laurent) | Lim of exp_laurent | Inf of exp_laurent | Sup of exp_laurent
```

4 Branch II: Super Homotopy Type Theory

This branch represents the central path of homotopical, cubical, and cohesive type systems, transitioning from pure dependent products to higher-dimensional geometry.

4.1 Henk: Pure Type Systems (PTS)

The initial object in LCC categories. Contains only Π and universes.

```
type term =
    | Var of string
    | Universe of int
    | Pi of string * term * term
    | Lam of string * term * term
    | App of term * term
```

4.2 Per: Martin-Löf Type Theory (MLTT-80)

MLTT-80 with W-types, base types 0, 1, 2, and cubical primitives.

```
type exp =
  | EPre of Z.t | EKan of Z.t
  | EVar of ident | EHole
  | EPi of exp * (ident * exp) | ELam of exp * (ident * exp) | EApp of exp * exp
  | ESig of exp * (ident * exp) | EPair of tag * exp * exp
  | EFst of exp | ESnd of exp | EField of exp * string
  | EId of exp | ERef of exp | EJ of exp
  | EPathP of exp | EPLam of exp | EAppFormula of exp * exp
  | EI | EDir of dir | EAnd of exp * exp | EOr of exp * exp | ENeg of exp
  | ETransp of exp * exp | EHComp of exp * exp * exp * exp
  | EPartial of exp | EPartialP of exp * exp | ESystem of exp System.t
  | ESub of exp * exp * exp | EInc of exp * exp | EOuc of exp
  | EEmpty | EIndEmpty of exp
  | EUnit | EStar | EIndUnit of exp
  | EBool | EFalse | ETrue | EIndBool of exp
  | EW of exp * (ident * exp) | ESup of exp * exp | EIndW of exp * exp * exp
```

4.3 Anders: Modal Cubical HoTT

CCHM cubical structure with Path, Glue, shape modality (ℑ), and higher inductive types (Coequ, Disc, Im, Fla).

```
type value =
  | VKan of Z.t | VPre of Z.t | Var of ident * value | VHole
  | VPi of value * clos | VLam of value * clos | VApp of value * value
  | VSig of value * clos | VPair of tag * value * value | VFst of value | VSnd of value
  | VId of value | VRef of value | VJ of value
  | VPathP of value | VPLam of value | VAppFormula of value * value
  | VI | VDir of dir | VAnd of value * value | VOr of value * value | VNeg of value
  | VTransp of value * value | VHComp of value * value * value * value
  | VGlue of value | VGlueElem of value * value * value | VUnglue of value * value * value
  | VEmpty | VIndEmpty of value | VUnit | VStar | VIndUnit of value
  | VBool | VFalse | VTrue | VIndBool of value
  | W of value * clos | VSup of value * value | VIndW of value * value * value
  | VNat | VZero | VSucc of value | VIndNat of value * value * value
  | VCoequ of value * value * value * value
  | VIm of value | VInf of value | VIndIm of value * value | VJoin of value
  | VFla of value | VFlaUnit of value | VFlaCounit of value | VIndFla of value * value
  ...
and clos = ident * (value -> value)
```

4.4 Urs: The Family of Layered Sublanguages

Urs is not a single monolithic type system, but rather a family of three layered sublanguages where each layer builds on top of the syntax, modalities, and constructors of the previous one:

Sublanguage	Modules	Primitives
I. Felix	`graded, flat, sharp, bose, fermi, tensor, smth, supsmth`	Bose/Fermi grades, $\flat$, $\sharp$, $A \otimes B$, SmthSet, SupSmthSet
II. Jack	`spectra, grpd, form, ku, diff, diffku`	Spectrum, Susp, Grpd, $\Omega^n(X)$, KU_G^τ, d, DiffKU
III. Urs	`conf, braid, qubit, linear`	Config^n, Braid$_n$, Qubit, $!A$

Remark 4.1 (The 3-Layer Architecture)**.** The sublanguages represent a progressive structure for formalizing mathematical and physical concepts:

1. **Felix**: Graded universes and tensors, group actions, and super-modality operators for geometric cohesion and differential structure—bosonic/fermionic grades, flat/sharp modalities.

2. **Jack**: Homotopical foundations via spectra and groupoids, supporting differential K-theory with differential forms, connections, and refined cohomology structures.

3. **Urs**: Configuration spaces and braids for Urs statistics, supporting Urs systems and linear types ($!A$) for resource-sensitive Urs programming.

Each sublanguage culminates in a system tailored to formalize superpoints ($\mathbb{R}^{m|n}$), supersymmetry, and equivariant structures.

4.4.1 Felix (Layer I)

Extends the homotopy core with graded universes, tensors, group actions, and cohesive modalities ($\flat$, $\sharp$, $\Im$) for synthetic supergeometry.

```
type grade = Bose | Fermi
type exp_coh =
  | Core of exp_hom
  | UniverseCoh of int * grade
  | SmthSet | Plot of int * exp_coh * exp_coh
  | Flat of exp_coh | Sharp of exp_coh | Shape of exp_coh
  | Bosonic of exp_coh | Tensor of exp_coh * exp_coh
  | SupSmthSet
```

4.4.2 Jack (Layer II)

Extends cohesion with stable homotopy (spectra, suspensions, wedges) and differential cohomology (forms, connections, and differential K-theory).

```
type exp_diff =
  | Coh of exp_coh
  | Grpd of int | Comp of int * exp_diff * exp_diff * exp_diff
  | Spectrum | Susp of exp_diff | Wedge of exp_diff * exp_diff
  | HomSpec of exp_diff * exp_diff | KU_G of exp_diff * exp_diff * exp_diff
  | Forms of int * exp_diff | Diff of int * exp_diff
  | DiffKU_G of exp_diff * exp_diff * exp_diff * exp_diff
```

4.4.3 Urs (Layer III)

The top layer, adding configuration spaces, braids, and linear resource types ($!A$) for Urs programming.

```
type exp_Urs =
  | DiffK of exp_diff
  | Qubit of exp_Urs * exp_Urs | Linear of exp_Urs
  | AppQubit of exp_Urs * exp_Urs * exp_Urs
  | FuseQubit of exp_Urs * exp_Urs * exp_Urs
  | Config of int * exp_Urs | Braid of int * exp_Urs
```

Remark 4.2 (STT $\to$ Urs: The Simplicial Connection)**.** Simplicial type theory (Ulrik/STT) provides a direct connection to Urs. The directed interval $\mathbb{I}^{\to}$, extension types, and modal Π in STT supply the ∞-categorical framework that Urs's Layer I (Homotopy) builds upon. By isolating the category of categories **Cat** as a reflective subuniverse in simplicial type theory, STT provides the required directed homotopy structures. Concretely, STT's directed interval becomes the path/homotopy backbone, while its modal/shape operations inform Urs's cohesive modalities ($\flat$, $\sharp$, $\Im$) in Layer II. This is represented by the STT arrow from Ulrik to Urs in the lattice diagram, which is *not* simply a factoring through Anders.

5 Branch III: Infinity Category Theory

This branch encompasses operational algebra and directed category theories, dealing with presentations of algebraic shapes and directed categorical structures.

5.1 Dan: Operational Algebra

Dan operates on a completely different level: it manipulates algebraic presentations (groups, simplicial sets, chain complexes) operationally. Its term language has no dependent types, no universes, and no Π/Σ. Instead it works with group words, permutations, matrices, face/degeneracy maps, and boundary operators.

```
type type_name =
  | Simplex | Group | Simplicial | Chain | Category | Monoid | Ring | Field
  | PermGroup | MatGroup | FpGroup | PcGroup | Module | Set | GSet
type term =
  | Id of string | Comp of term * term | Inv of term | Pow of term * superscript
  | E | Matrix of int list list | Add of term * term | Mul of term * term
  | Perm of int list | Cycle of int list list
  | PermGroup of term list | MatGroup of term list * int * int
  | FpGroup of string list * term list | PcGroup of string list * term list
  | Face of superscript * term | Deg of superscript * term | Boundary of term
  | Act of term * term | Hom of term * term | RelEq of term * term
type hypothesis =
  | Decl of string list * type_name
  | Equality of string * term * term
  | Mapping of string * term * term
  | Presentation of string * term | Relation of term
  | Action of string * term * term
  | Orbit of term * term * term list | Stabilizer of term * term * term
```

5.2 Mike: Directed 1-Category Type Theory

A *linear* type theory with a bidirectional type checker enforcing quadraticality (each category variable occurs exactly once covariantly and once contravariantly).

```
type cat = CVar of name | COp of cat | CProd of cat * cat
type m_type =
  | MHom of cat * cat_term * cat_term | MTensor of m_type * m_type
  | MCoend of cat * name * m_type | MEnd of cat * name * m_type
  | MFunc of m_type * m_type | MApp of name * cat_term list
type m_term =
  | MTId of cat_term
  | MTJ of m_type * name * name * name * m_term * cat_term * cat_term * m_term
  | MTJCov of m_type * name * m_term * cat_term * m_term
  | MTJContra of m_type * name * m_term * cat_term * m_term
  | MTTensorIntro of m_term * m_term | MTTensorElim of name * name * m_term * m_term
  | MTCoendIntro of name * name * name * m_term
  | MTCoendElim of name * name * m_term * m_term
  | MTEndIntro of name * m_term
  | MTEndElim of name * name * name * m_term * m_term
  | MTFuncIntro of name * m_type * m_term | MTFuncElim of m_term * m_term
  | MTVar of name
```

5.3 Ulrik: Simplicial ∞-Categories

Extends Mike's foundation with directed interval $\mathbb{I}$, shape inclusions ($\varphi \subseteq \psi$), extension types, modal Π, and twisted arrow categories (A^{tw}).

To construct a type theory for synthetic category theory, one might hope to interpret type theory in the category of categories (∞-categories or otherwise) to ensure that types realize categories. However, the category **Cat** of small

categories is too poorly behaved to form a model of Martin-Löf Type Theory (MLTT). Instead, Riehl and Shulman (2017) embed **Cat** as a reflective subcategory in the category of (∞-)presheaves on the simplex category $\widehat{\Delta}$, which is sufficiently rich to model HoTT. Simplicial Type Theory (STT) then axiomatizes a portion of $\widehat{\Delta}$ to isolate **Cat** as a reflective subuniverse within the type theory.

```
type exp =
  | EUniv | EVar of name | EPi of exp * (name * exp) | ELam of (name * exp option) * exp
  | EApp of exp * exp | ESig of exp * (name * exp) | EPair of exp * exp
  | EFst of exp | ESnd of exp | EId of exp * exp * exp | ERef of exp
  | EIDir | EZeroDir | EOneDir | ELeq of exp * exp | EShapeInc of exp * exp
  | ESystem of (exp * exp) list | EExt of exp * exp * exp
  | EModalPi of exp * (name * exp) | EModalLam of (name * exp) * exp
  | EModalApp of exp * exp | ETw of exp | ETwPi0 of exp | ETwPi1 of exp
  | EJoin of exp * exp | EMeet of exp * exp | ENeg of exp
  | EJ of exp * name * name * name * exp * exp * exp * exp
  | EJCov of exp * name * exp * exp * exp | EJContra of exp * name * exp * exp * exp
```

6 Cohomology of Programming Languages

Each language's syntax is analyzed *individually* as a CW-complex.

For a formal language $\mathcal{O}_x$, we associate a chain complex of abelian groups (free $\mathbb{Z}$-modules) $C_*(\mathcal{O}_x)$ constructed from the syntax and reduction/conversion semantics. This construction models the syntactic space as a cell complex (CW-complex) where paths and higher homotopies encode the computational rules.

Definition 6.1 (Syntactic CW-Complex)**.** For a language $\mathcal{O}_x$ with term syntax T generated by a signature Σ, the syntactic cell complex $C_*(\mathcal{O}_x)$ is defined in low dimensions by:

- **0-cells ($C_0(\mathcal{O}_x)$)**: The free $\mathbb{Z}$-module generated by the set of basic syntactic constructors $c \in \Sigma$ (e.g., `Var`, `Pi`, `Lam`, `App` for Henk). Any term $t \in T$ has an associated constructor-occurrence representation:

$$[t] = \sum_{c \in \Sigma} \text{count}(c, t) \cdot c \quad \in C_0(\mathcal{O}_x)$$

  where $\text{count}(c, t)$ is the number of times the constructor c appears in the syntax tree of t.

- **1-cells ($C_1(\mathcal{O}_x)$)**: The free $\mathbb{Z}$-module generated by the rewriting and reduction rules $r \in \mathcal{R}$ of the language (e.g., β-reductions, evaluation steps). Each 1-cell $r : t \to t'$ represents a directed rewrite path from a redex term t to a contractum term t'.

- **2-cells ($C_2(\mathcal{O}_x)$)**: The free $\mathbb{Z}$-module generated by equivalence relations, critical pairs, confluence diagrams, and conversion equations (e.g., η-conversions, commuting conversions). A 2-cell e represents a coherence condition or a closed loop of rewrite steps.

- **Higher cells ($C_k(\mathcal{O}_x)$ for $k \geq 3$)**: Coherence relations between equations, such as Mac Lane pentagon relations, path-type univalence coherences (in Anders), or quadraticality constraints (in Mike).

Definition 6.2 (Boundary Operators)**.** The boundary operators $\partial_k : C_k(\mathcal{O}_x) \to C_{k-1}(\mathcal{O}_x)$ are defined as follows:

1. The 1-boundary operator $\partial_1 : C_1(\mathcal{O}_x) \to C_0(\mathcal{O}_x)$ is the linear map determined by the net change in constructor counts caused by a rewrite rule $r : t \to t'$:

$$\partial_1(r) = [t'] - [t]$$

2. The 2-boundary operator $\partial_2 : C_2(\mathcal{O}_x) \to C_1(\mathcal{O}_x)$ is determined by the boundary of confluence diagrams or conversion loops. If a 2-cell e represents a confluence diagram from a term s reducing to s' via two different

paths of rewrites $p_1 = (r_{1,1}, \ldots, r_{1,m})$ and $p_2 = (r_{2,1}, \ldots, r_{2,n})$, the boundary is:

$$\partial_2(e) = \sum_{j=1}^{m} r_{1,j} - \sum_{k=1}^{n} r_{2,k}$$

Since both paths p_1 and p_2 start at s and end at s', the net constructor changes must match:

$$\partial_1(p_1) = [s'] - [s] = \partial_1(p_2)$$

Thus:

$$\partial_1(\partial_2(e)) = \partial_1(p_1) - \partial_1(p_2) = 0$$

This proves that $\partial_1 \circ \partial_2 = 0$, establishing a valid chain complex.

Definition 6.3 (Syntactic Cohomology)**.** Let $C_*(\mathcal{O}_x)$ be the syntactic chain complex of $\mathcal{O}_x$ and let G be an abelian group of coefficients. The cochain complex $C^*(\mathcal{O}_x; G) = \mathrm{Hom}(C_*(\mathcal{O}_x), G)$ has coboundary operators $\delta^k : C^k(\mathcal{O}_x; G) \to C^{k+1}(\mathcal{O}_x; G)$ defined by $\delta^k(\phi) = \phi \circ \partial_{k+1}$. The syntactic cohomology groups are:

$$H^k(\mathcal{O}_x; G) = \ker(\delta^k)/\mathrm{im}(\delta^{k-1})$$

6.1 Physical Interpretations of Cohomology Groups

The cohomology groups $H^k(\mathcal{O}_x; G)$ measure fundamental properties of the syntax and semantics of $\mathcal{O}_x$:

- $H^0(\mathcal{O}_x; G)$: Measures the *syntactic dimension* or independent constructor classes. A class in H^0 is a function $f : C_0 \to G$ such that for any rewrite $r : t \to t'$, $f([t]) = f([t'])$. Thus, H^0 captures invariants preserved under computation (such as type preservation or conserved weights of terms).

- $H^1(\mathcal{O}_x; G)$: Measures *normalization obstructions*. An element in H^1 is represented by a 1-cocycle (an assignment of weights to rewrites that sums to zero on closed loops) modulo those coming from constructor valuations. Non-trivial H^1 classes indicate non-confluent rewrites, cycles of reductions (non-termination), or irreversible compilation steps.

- $H^2(\mathcal{O}_x; G)$: Measures *coherence obstructions*. It captures the obstructions to filling confluence diagrams with higher-dimensional conversion cells. A non-trivial class in H^2 represents a critical pair (or confluence loop) that cannot be closed by conversions or equivalence cells, pointing to deep semantic mismatches or failures of univalence/coherence.

6.2 Examples across the Branches

Example 6.4 (Branch I/II: Constructive & Homotopical Spine)**.**

- **Henk ($\mathcal{O}_{\Pi}$)**: The core type theory.

  $$C_0 = \{\texttt{Var}, \texttt{Universe}, \texttt{Pi}, \texttt{Lam}, \texttt{App}\}$$

  The 1-cells are generated by β-reduction:

  $$r_\beta : \texttt{App}(\texttt{Lam}(x, A, t), u) \to t[u/x]$$

  $\partial_1(r_\beta) = [t[u/x]] - (\texttt{App} + \texttt{Lam} + [t] + [u])$. The 2-cells consist of η-conversions:

  $$e_\eta : \texttt{Lam}(x, A, \texttt{App}(t, x)) \equiv_\eta t$$

  Since Henk is strongly normalizing and confluent (Church-Rosser), the higher cohomology $H^k(\mathcal{O}_{\Pi}; \mathbb{Z})$ for $k \geq 1$ is trivial, reflecting a contractible syntactic space.

- **Anders ($\mathcal{O}_{\text{Anders}}$)**: Adds univalence and Glue types. The univalence axiom represents a path-space equivalence between isomorphic types. This introduces higher-dimensional coherences (Glue cells) acting as 3-cells, preventing non-trivial 2-dimensional obstructions.

- **Urs ($\mathcal{O}_{\text{Urs}}$)**: Adds modalities ($\flat, \sharp$) and differential structures (forms, connections). The syntactic CW-complex of $\mathcal{O}_{\text{Urs}}$ is enriched with topological and differential cells, yielding non-trivial H^k corresponding to de Rham cohomology and Chern-Weil invariants of smooth/differential type structures.

Example 6.5 (Branch III: Operational & Directed Branch)**.**

- **Dan ($\mathcal{O}_{\text{Dan}}$)**: Operational algebra. Here, term syntax encodes algebraic operations (e.g., face maps ∂_i, degeneracies s_i). The cell complex $C_*(\mathcal{O}_{\text{Dan}})$ is isomorphic to the standard simplicial chain complex of the underlying category/algebraic presentation.

- **Mike ($\mathcal{O}_{\text{Mike}}$)**: Directed category theory. 1-cells are non-reversible functors/morphisms, meaning paths are directed. The boundary operator ∂_1 tracks directed change. Quadraticality constraints act as 2-cells that model the non-commutative relation profiles of monoidal/directed types.

- **Ulrik ($\mathcal{O}_{\text{Ulrik}}$)**: Simplicial ∞-categories. Generalizes Mike's 1-categories to directed simplicial spaces. The higher cells (C_k for $k \geq 2$) represent the Segal completeness conditions and Rezk completeness conditions, ensuring that the directed paths compose coherently up to higher homotopies.

7 Spectral Sequence of Language Categories

Unlike a simple linear filtration, the language category $\mathcal{L}$ is organized as a *lattice* with multiple join-paths. The spectral sequence is defined over the partial order of syntactic subcategories:

$$E_1^{p,q} = H^q(\mathcal{O}_p) \implies H^{p+q}(\mathcal{L})$$

p	**Language**	**Branch**	**Primitives added**	$\lvert C_0\rvert$
0	Henk	CTT	Π, U^n	5
1	Frank	CTT	+ Ind	8
2a	Christine	CTT	+ Σ, Id	14
2b	Per	CTT	+ Σ, Id, W, 0, 1, 2	26
3	Anders	CTT	+ Path, I, Glue, HComp, HIT	40+
4a	Felix	CTT/Modal	+ $\flat$, $\sharp$, $\Im$, $\otimes$, Plot	10
4b	Jack	CTT/Modal	+ Spectra, Susp, Grpd, Forms, DiffKU	12
4c	Urs	CTT/Modal	+ Qubit, Linear, Config, Braid	7
—	Dan	Algebra	Groups, Simplices, Chains, Boundaries	20
—	Mike	Directed	Hom, $\otimes$, $\int$, $\prod$, $\multimap$	13
—	Ulrik	Directed	+ $\mathbb{I}^{\to}$, Ext, Tw, $\leq$	25

Table 1: The language lattice: filtration indices and constructor counts. Note the split at $p = 2$ (Christine/Per), the four sublanguages of Urs at $p = 4$ (Homotopy, SupSmth, DiffK, Urs), and the separate branch indices for Dan, Mike, and Ulrik.

Definition 7.1 (Morphisms in $\mathcal{L}$)**.** Morphisms $f : \mathcal{O}_x \to \mathcal{O}_y$ are classified as:

1. $\mathcal{F}_{\text{enrich}}$: Enrichment (type inference, elaboration).
2. $\mathcal{F}_{\text{simp}}$: Simplification (type erasure, extraction).
3. $\mathcal{F}_{\text{trans}}$: Translation (compilation across branches).
4. $\mathcal{F}_{\text{eval}}$: Evaluation (normalization to values).
5. $\mathcal{F}_{\text{norm}}$: Normalization ($\beta\eta$-reduction).
6. $\mathcal{F}_{\text{certify}}$: Certification (proof checking).

Cross-branch morphisms (e.g. **Lift**: Dan $\to$ Mike, **Core**: Ulrik $\to$ Anders) are structure-changing and do not preserve the term type.

7.1 Functorial Conversions and Cross-Branch Morphisms

The language category $\mathcal{L}$ is not merely a set of objects; the relationships between the branches are mediated by structured functorial conversions. The primary conversions, their statuses, and associated information losses are detailed in Table 2.

Functor / Conversion	Source	Target	Status	Semantics & Information Loss
Lift	Dan	Mike	Yes	Semantical lifting of presentation generators and equations to 1-categorical structures.
Forget	Mike	Dan	Partial	Truncates categories back to operation-based presentation generators.
Free	Mike	Ulrik	Yes	Fully faithful embedding of directed 1-categories into directed ∞-categories.
Trunc	Ulrik	Mike	No	Cannot be done functorially without losing higher-dimensional coherence.
Core	Ulrik	Anders	Yes	Groupoid Core: discards directed path structure, retaining only the sub-groupoid of isomorphisms.
Disc	Anders	Ulrik	Yes	Discrete embedding: interprets ∞-groupoids as discrete ∞-categories via disc modality.
Forget	Anders	Mike	Partial	1-category approximation: projects higher paths onto 1-category hom-sets.
Groupoid	Mike	Anders	Yes	Forgetful functor from directed 1-categories to spaces of isomorphism paths.
Lift (Composite)	Dan	Ulrik	Yes	Composition of **Lift** and **Free**: maps operations to Rezk ∞-categories.
Forget	Ulrik	Dan	No	Directed simplicial spaces are too rich to map back to discrete operations directly.
Forget	Anders	Dan	No	Undirected spaces cannot be functorially mapped to discrete presentation equations.
Modal	Anders	Felix	Yes	Extends cubical HoTT with cohesive modalities ($\flat$, $\sharp$, $\Im$) and super-grades.
Diff	Felix	Jack	Yes	Stable extension to spectra, suspensions, forms, and differential K-theory.
Urs	Jack	Urs	Yes	Extends differential K-theory with configuration spaces, braids, and linear types.

Table 2: Primary conversions and functors between the language layers.

7.1.1 Deep Dive: The Lift Functor (Dan $\to$ Mike/Ulrik)

The **Lift** functor acts as a semantical compilation bridge that transports combinatorial presentations from the operational layer **Dan** to the directed synthetic type theories of **Mike** and **Ulrik**. Rather than a simple syntax translator, it elaborates discrete presentations into rich categorical models in four distinct phases:

1. **Signature Ingestion**: Reads the generator set, face maps (∂_i), degeneracy maps, and algebraic presentation relations directly from the **Dan** syntax.

2. **Semantical Lifting**: Maps the ingested signature into category-theoretic structures, translating generator equations into functors, categories, and sheaves.

3. **Elaboration to STT**: Translates composition equations to identity paths (`EId`) in **Ulrik**. The directed interval $\mathbb{I}^{\to}$ is employed to model the domains and boundary conditions.

4. **Metatheory Absorption**: Automatically equips the lifted object with the abstract category-theoretic library of **Ulrik** (including Yoneda lemmas, limits, adjunctions, and Kan extensions).

7.1.2 Deep Dive: The Groupoid Core (Ulrik → Anders)

The translation from **Ulrik** to **Anders** is a structural forgetful conversion that extracts the underlying homotopy type (the ∞-groupoid of isomorphisms) from a directed ∞-category. Since the simplicial directed interval $\mathbb{I}^{\to}$ and directed hom-types ($x \to y$) of **Ulrik** do not exist in the cubical undirected world of **Anders**, the functor operates as follows:

- Discards directed arrows, keeping only the sub-groupoid A^{core} containing morphisms with valid inverses.
- Discards the Segal and Rezk completeness conditions, transforming directed category composition into symmetric path composition.
- Replaces the directed interval $\mathbb{I}^{\to}$ with the cubical symmetric interval I (with connection and reversal operations).

7.1.3 Case Study: Möbius Strip Representation

To illustrate the conversions, Table 3 compares the representation of a Möbius Strip across the three primary layers (**Dan**, **Ulrik**, and **Anders**).

Aspect	**Dan (Operational)**	**Ulrik (Simplicial STT)**	**Anders (Cubical HoTT)**
Framework	Computer algebra presentations	Simplicial Type Theory with $\mathbb{I}^{\to}$	Cubical HoTT with modal operators
Mathematical Type	Combinatorial simplex complex with boundary relations	Rezk ∞-category built from simplicial diagrams	Univalent fiber bundle: $\sum_{x:S^1} \text{Moebius}(x)$
Twist Encoding	Equations on face maps (∂_i)	Directed paths over $\mathbb{I}^{\to}$ satisfying Segal conditions	Univalent path transport (`ua`) using boolean negation (`true` $\equiv$ `false`)
Homotopy Type	Contractible local complex ($*$)	∞-category whose core homotopy type is S^1	Homotopy equivalence to the circle S^1
Abstraction Level	Combinatorial building blocks	Global synthetic ∞-category	Global synthetic univalent bundle
Functor Action	Ingested by **Lift**	Lifted to directed ∞-category	Extracted via **Core** as undirected ∞-groupoid
Information Kept	Full combinatorial generators	Full directed structure	Homotopy skeleton; directed composition is lost

Table 3: Comparison of the Möbius Strip representation across the layers.